Restaurant Ordering and Kitchen Management System

Complete Project Documentation, Database Design, API Reference, and Workflow

Stack: React + Tailwind + Express + SQL + Socket.IO + Razorpay

1. Project Overview

This system handles restaurant ordering from a customer kiosk, sends orders to the kitchen, lets the chef accept and estimate preparation time, updates the customer in real time, assigns the correct waiter when the food is ready, handles cash or online payment, and stores reviews after completion.

2. Problem It Solves

Restaurants lose time and accuracy when orders are taken manually. Orders get delayed, kitchen communication breaks, waiter assignment becomes confusing, and customers do not know the status of their food.

3. Main Roles

Customer places the order and tracks it. Chef accepts the order and updates food status. Waiter receives assignment, serves food, and collects cash when needed. Admin manages staff, menu, tables, and reports.

4. Order Status Flow

placed -> accepted -> preparing -> prepared -> assigned_to_waiter -> served -> paid -> reviewed

5. Why SQL Database

The system has strong relationships between orders, items, customizations, payments, and staff assignments. A relational database keeps this structure clean and consistent.

- placed: customer has submitted the order
- accepted: chef has accepted and given ETA
- preparing: chef is cooking the food
- prepared: food is ready in the kitchen
- assigned_to_waiter: system assigns a waiter
- served: waiter serves the food to the table
- paid: payment is completed
- reviewed: customer submitted feedback

6. Database Design

Table	Purpose	Main columns	Why it exists
roles	Stores user roles	role_id, name	Supports access control
users	Stores staff users	user_id, name, phone, password_hash, role_id, is_active, created_at	Single source for all staff accounts
restaurant_tables	Stores physical tables	table_id, table_number,	Maps orders to tables

		capacity, status	
categories	Stores menu categories	category_id, name	Organizes menu items
menu_items	Stores menu items	item_id, name, description, price, category_id, is_available, created_at	Main food catalog
item_customizations	Stores customization types	customization_id, item_id, name, type	Flexible item-level options
customization_options	Stores allowed values	option_id, customization_id, value, price_modifier	Supports price and choice variations
orders	Stores order header	order_id, table_id, customer_name, customer_phone, status, payment_method, payment_status, total_amount, eta_minutes, created_at, updated_at	Heart of the workflow
order_items	Stores items in an order	order_item_id, order_id, item_id, quantity, price_at_order_time	Keeps order items normalized
order_item_customizations	Stores selected customizations	id, order_item_id, customization_id, option_id	Supports multiple options per item
order_status_logs	Stores every status change	log_id, order_id, status, changed_by, timestamp	Audit history
waiter_assignments	Stores waiter assignment	assignment_id, order_id, waiter_id, assigned_at, acknowledged, served_at	Tracks assignment and acknowledgement
payments	Stores payment info	payment_id, order_id, method, amount, status, transaction_id, paid_at	Separates payment from order data
reviews	Stores customer feedback	review_id, order_id, rating, comment, created_at	Collects ratings after completion
notifications	Stores notifications	notification_id, user_id, order_id, type, message, is_read, created_at	Stores read/unread notification history

7. API Conventions

- Base URL: /api
- Success responses use success=true
- Validation or failed actions return success=false with a message
- Use JWT in Authorization header for staff routes
- Only authorized roles can access protected endpoints

8. Authentication APIs

These APIs handle login, registration, profile fetch, and logout for staff accounts.

POST /api/auth/register

Purpose: Create a staff account for admin, chef, or waiter.

Request body:

```
{
  "name": "Rahul",
  "phone": "9876543210",
  "password": "123456",
  "role": "waiter"
}
```

Response 201:

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "userId": 12,
    "name": "Rahul",
    "phone": "9876543210",
    "role": "waiter"
  }
}
```


Common errors: 409 if phone already exists; 400 if role or fields are invalid.

POST /api/auth/login

Purpose: Login a staff user and return a JWT token.

Request body:


```
{
  "phone": "9876543210",
  "password": "123456"
}
```


Response 200:


```
{
  "success": true,
  "message": "Login successful",
  "token": "jwt_token_here",
  "data": {
    "userId": 12,
    "name": "Rahul",
    "role": "waiter"
  }
}
```


Common errors: 401 if password is wrong; 404 if user not found.

GET /api/auth/me

Purpose: Return the current logged-in staff user.

Response 200:


```
{
  "success": true,
  "data": {
    "userId": 12,
    "name": "Rahul",
    "phone": "9876543210",
    "role": "waiter"
  }
}
```


Common errors: 401 if token is missing or expired.

POST /api/auth/logout

Purpose: Log out the current staff user.

Response 200:


```
{
  "success": true,
```

```
  "message": "Logged out successfully"
}
```

9. Role APIs

These APIs manage role records used for access control.

GET /api/roles

Purpose: Get all roles.

Response 200:

```
{
  "success": true,
  "data": [
    { "roleId": 1, "name": "admin" },
    { "roleId": 2, "name": "chef" },
    { "roleId": 3, "name": "waiter" }
  ]
}
```

POST /api/roles

Purpose: Create a new role.

Request body:

```
{ "name": "cashier" }
```

Response 201:

```
{
  "success": true,
  "message": "Role created successfully",
  "data": { "roleId": 4, "name": "cashier" }
}
```

PATCH /api/roles/:roleId

Purpose: Update a role name.

Request body:

```
{ "name": "senior waiter" }
```

Response 200:

```
{ "success": true, "message": "Role updated successfully" }
```

DELETE /api/roles/:roleId

Purpose: Delete a role.

Response 200:

```
{ "success": true, "message": "Role deleted successfully" }
```

10. User APIs

Admin uses these APIs to manage staff users.

GET /api/users

Purpose: Get all staff users.

Response 200:

```
{
  "success": true,
```

```
  "data": [
    {
      "userId": 12,
      "name": "Rahul",
      "phone": "9876543210",
      "role": "waiter",
      "isActive": true
    }
  ]
}
```

GET /api/users/:userId

Purpose: Get one staff user.

Response 200:

```
{
  "success": true,
  "data": {
    "userId": 12,
    "name": "Rahul",
    "phone": "9876543210",
    "role": "waiter"
  }
}
```

POST /api/users

Purpose: Create a staff user.

Request body:

```
{
  "name": "Rahul",
  "phone": "9876543210",
  "password": "123456",
  "roleId": 3
}
```

Response 201:

```
{ "success": true, "message": "User created successfully" }
```

PATCH /api/users/:userId

Purpose: Update a staff user.

Request body:

```
{
  "name": "Rahul Sharma",
  "phone": "9876543211",
  "roleId": 3,
  "isActive": true
}
```

Response 200:

```
{ "success": true, "message": "User updated successfully" }
```

DELETE /api/users/:userId

Purpose: Delete a staff user.

Response 200:

```
{ "success": true, "message": "User deleted successfully" }
```

11. Table APIs

These APIs manage the physical restaurant tables.

GET /api/tables

Purpose: Get all restaurant tables.

Response 200:

```
{
  "success": true,
  "data": [
    { "tableId": 1, "tableNumber": "T1", "capacity": 4, "status": "available" }
  ]
}
```

GET /api/tables/:tableId

Purpose: Get one table.

Response 200:

```
{
  "success": true,
  "data": {
    "tableId": 1,
    "tableNumber": "T1",
    "capacity": 4,
    "status": "occupied"
  }
}
```

POST /api/tables

Purpose: Create a table.

Request body:

```
{
  "tableNumber": "T10",
  "capacity": 4
}
```

Response 201:

```
{
  "success": true,
  "message": "Table created successfully",
  "data": {
    "tableId": 10,
    "tableNumber": "T10",
    "capacity": 4,
    "status": "available"
  }
}
```

PATCH /api/tables/:tableId

Purpose: Update a table.

Request body:

```
{
  "tableNumber": "T10",
  "capacity": 6
}
```

Response 200:

```
{ "success": true, "message": "Table updated successfully" }
```

PATCH /api/tables/:tableId/status

Purpose: Update table status.

Request body:

```
{ "status": "occupied" }
```

Response 200:

```
{ "success": true, "message": "Table status updated" }
```

DELETE /api/tables/:tableId

Purpose: Delete a table.

Response 200:

```
{ "success": true, "message": "Table deleted successfully" }
```

12. Category APIs

These APIs group menu items into categories.

GET /api/categories

Purpose: Get all categories.

Response 200:

```
{
  "success": true,
  "data": [{ "categoryId": 1, "name": "Main Course" }]
}
```

GET /api/categories/:categoryId

Purpose: Get one category.

Response 200:

```
{
  "success": true,
  "data": { "categoryId": 1, "name": "Main Course" }
}
```

POST /api/categories

Purpose: Create a category.

Request body:

```
{ "name": "Starter" }
```

Response 201:

```
{
  "success": true,
  "message": "Category created successfully",
  "data": { "categoryId": 2, "name": "Starter" }
}
```

PATCH /api/categories/:categoryId

Purpose: Update a category.

Request body:

```
{ "name": "Snacks" }
```

Response 200:

```
{ "success": true, "message": "Category updated successfully" }
```

DELETE /api/categories/:categoryId

Purpose: Delete a category.

Response 200:

```
{ "success": true, "message": "Category deleted successfully" }
```

13. Menu Item APIs

These APIs manage the food catalog.

GET /api/menu-items

Purpose: Get all menu items.

Response 200:

```
{
  "success": true,
  "data": [
    {
      "itemId": 1,
      "name": "Paneer Butter Masala",
      "description": "Creamy paneer dish",
      "price": 220,
      "categoryId": 1,
      "isAvailable": true
    }
  ]
}
```

GET /api/menu-items/:itemId

Purpose: Get one menu item.

Response 200:

```
{
  "success": true,
  "data": {
    "itemId": 1,
    "name": "Paneer Butter Masala",
    "description": "Creamy paneer dish",
    "price": 220,
    "categoryId": 1,
    "isAvailable": true
  }
}
```

POST /api/menu-items

Purpose: Create a menu item.

Request body:

```
{
  "name": "Paneer Butter Masala",
  "description": "Creamy paneer dish",
  "price": 220,
  "categoryId": 1,
```

```
  "isAvailable": true
}
```

Response 201:

```
{
  "success": true,
  "message": "Menu item created successfully",
  "data": { "itemId": 1, "name": "Paneer Butter Masala" }
}
```

PATCH /api/menu-items/:itemId

Purpose: Update a menu item.

Request body:

```
{
  "name": "Paneer Butter Masala Full",
  "description": "Creamy paneer dish with extra gravy",
  "price": 240,
  "categoryId": 1,
  "isAvailable": true
}
```

Response 200:

```
{ "success": true, "message": "Menu item updated successfully" }
```

PATCH /api/menu-items/:itemId/availability

Purpose: Update item availability.

Request body:

```
{ "isAvailable": false }
```

Response 200:

```
{ "success": true, "message": "Menu item availability updated" }
```

DELETE /api/menu-items/:itemId

Purpose: Delete a menu item.

Response 200:

```
{ "success": true, "message": "Menu item deleted successfully" }
```

14. Menu Customization APIs

These APIs manage customization rules for menu items.

GET /api/menu-items/:itemId/customizations

Purpose: Get all customizations for one menu item.

Response 200:

```
{
  "success": true,
  "data": [{ "customizationId": 1, "name": "Spice Level", "type": "dropdown" }]
}
```

POST /api/menu-items/:itemId/customizations

Purpose: Create a customization for a menu item.

Request body:

```
{ "name": "Spice Level", "type": "dropdown" }
```

Response 201:

```
{
  "success": true,
  "message": "Customization created successfully",
  "data": { "customizationId": 1, "name": "Spice Level" }
}
```

PATCH /api/customizations/:customizationId

Purpose: Update a customization.

Request body:

```
{ "name": "Oil Level", "type": "dropdown" }
```

Response 200:

```
{ "success": true, "message": "Customization updated successfully" }
```

DELETE /api/customizations/:customizationId

Purpose: Delete a customization.

Response 200:

```
{ "success": true, "message": "Customization deleted successfully" }
```

POST /api/customizations/:customizationId/options

Purpose: Add an option to a customization.

Request body:

```
{ "value": "Medium", "priceModifier": 0 }
```

Response 201:

```
{
  "success": true,
  "message": "Option added successfully",
  "data": { "optionId": 5, "value": "Medium" }
}
```

GET /api/customizations/:customizationId/options

Purpose: Get all options of a customization.

Response 200:

```
{
  "success": true,
  "data": [{ "optionId": 5, "value": "Medium", "priceModifier": 0 }]
}
```

PATCH /api/customization-options/:optionId

Purpose: Update a customization option.

Request body:

```
{ "value": "High", "priceModifier": 10 }
```

Response 200:

```
{ "success": true, "message": "Customization option updated successfully" }
```

DELETE /api/customization-options/:optionId

Purpose: Delete a customization option.

Response 200:

```
{ "success": true, "message": "Customization option deleted successfully" }
```

15. Order APIs

These APIs manage the full order lifecycle.

POST /api/orders

Purpose: Create a new order from the customer kiosk.

Request body:

```
{
 "tableId": 1,
 "customerName": "Aman",
 "customerPhone": "9876543210",
 "paymentMethod": "cash",
 "items": [
  {
   "itemId": 1,
   "quantity": 2,
   "customizations": [{ "customizationId": 1, "optionId": 2 }]
  }
 ]
}
```

Response 201:

```
{
 "success": true,
 "message": "Order placed successfully",
 "data": {
  "orderId": 101,
  "status": "placed",
  "paymentStatus": "pending",
  "totalAmount": 440
 }
}
```

GET /api/orders

Purpose: Get all orders for kitchen, waiter, or admin dashboard.

Response 200:

```
{
 "success": true,
 "data": [
  {
   "orderId": 101,
   "customerName": "Aman",
   "tableNumber": "T1",
   "status": "preparing",
   "paymentMethod": "cash",
   "paymentStatus": "pending"
  }
 ]
}
```

GET /api/orders/:orderId

Purpose: Get one full order.

Response 200:

```
{
  "success": true,
  "data": {
    "orderId": 101,
    "customerName": "Aman",
    "tableNumber": "T1",
    "status": "prepared",
    "etaMinutes": 15,
    "items": [
      { "orderItemId": 1, "name": "Paneer Butter Masala", "quantity": 2 }
    ]
  }
}
```


PATCH /api/orders/:orderId/customer

Purpose: Update customer name or phone for an order.

Request body:


```
{
  "customerName": "Aman Kumar",
  "customerPhone": "9999999999"
}
```


Response 200:


```
{ "success": true, "message": "Customer details updated successfully" }
```


PATCH /api/orders/:orderId/accept

Purpose: Chef accepts the order and sets ETA.

Request body:


```
{ "etaMinutes": 20 }
```


Response 200:


```
{
  "success": true,
  "message": "Order accepted",
  "data": { "status": "accepted", "etaMinutes": 20 }
}
```


PATCH /api/orders/:orderId/start-preparing

Purpose: Move order to preparing state.

Response 200:


```
{ "success": true, "message": "Order moved to preparing" }
```


PATCH /api/orders/:orderId/mark-prepared

Purpose: Mark the order as prepared.

Response 200:


```
{ "success": true, "message": "Order marked as prepared" }
```


PATCH /api/orders/:orderId/cancel

Purpose: Cancel an order.

Request body:


```
{ "reason": "Customer changed mind" }
```


Response 200:


```
{ "success": true, "message": "Order cancelled successfully" }
```

DELETE /api/orders/:orderId

Purpose: Delete an order record.

Response 200:

```
{ "success": true, "message": "Order deleted successfully" }
```

16. Order Item APIs

These APIs manage items inside an order.

GET /api/orders/:orderId/items

Purpose: Get all items for an order.

Response 200:

```
{
  "success": true,
  "data": [
    {
      "orderItemId": 1,
      "itemId": 1,
      "name": "Paneer Butter Masala",
      "quantity": 2,
      "priceAtOrderTime": 220
    }
  ]
}
```

POST /api/orders/:orderId/items

Purpose: Add a new item to an existing order.

Request body:

```
{ "itemId": 1, "quantity": 2 }
```

Response 201:

```
{ "success": true, "message": "Order item added successfully" }
```

PATCH /api/order-items/:orderItemId

Purpose: Update an order item quantity.

Request body:

```
{ "quantity": 3 }
```

Response 200:

```
{ "success": true, "message": "Order item updated successfully" }
```

DELETE /api/order-items/:orderItemId

Purpose: Delete an order item.

Response 200:

```
{ "success": true, "message": "Order item deleted successfully" }
```

POST /api/order-items/:orderItemId/customizations

Purpose: Attach customization to order item.

Request body:

```
{ "customizationId": 1, "optionId": 2 }
```

Response 201:

```
{ "success": true, "message": "Customization added to order item" }
```

DELETE /api/order-item-customizations/:id

Purpose: Remove customization from order item.

Response 200:

```
{ "success": true, "message": "Order item customization removed successfully" }
```

17. Order Status Log APIs

These APIs expose and store the order history.

GET /api/orders/:orderId/status-logs

Purpose: Get the full status history for an order.

Response 200:

```
{
 "success": true,
 "data": [
  {
   "logId": 1,
   "status": "placed",
   "changedBy": "customer",
   "timestamp": "2026-05-03T10:00:00Z"
  }
 ]
}
```

POST /api/orders/:orderId/status-logs

Purpose: Add a status log entry manually.

Request body:

```
{ "status": "prepared", "changedBy": 12 }
```

Response 201:

```
{ "success": true, "message": "Status log added successfully" }
```

18. Waiter Assignment APIs

These APIs assign, acknowledge, and reassign waiters.

POST /api/orders/:orderId/assign-waiter

Purpose: Assign the next available waiter automatically.

Response 200:

```
{
 "success": true,
 "message": "Waiter assigned successfully",
 "data": {
  "assignmentId": 55,
  "waiterId": 5,
  "waiterName": "Suresh"
 }
}
```

GET /api/orders/:orderId/assignment

Purpose: Get waiter assignment for an order.

Response 200:

```
{
  "success": true,
  "data": {
    "assignmentId": 55,
    "waiterId": 5,
    "waiterName": "Suresh",
    "acknowledged": true
  }
}
```

GET /api/waiter-assignments

Purpose: Get all waiter assignments.

Response 200:

```
{ "success": true, "data": [] }
```

PATCH /api/waiter-assignments/:assignmentId/acknowledge

Purpose: Waiter acknowledges the assignment.

Response 200:

```
{ "success": true, "message": "Waiter acknowledged assignment" }
```

PATCH /api/waiter-assignments/:assignmentId/serve

Purpose: Mark order served from waiter side.

Response 200:

```
{ "success": true, "message": "Order marked as served" }
```

PATCH /api/waiter-assignments/:assignmentId/reassign

Purpose: Reassign order to a different waiter.

Request body:

```
{ "newWaiterId": 6 }
```

Response 200:

```
{ "success": true, "message": "Waiter reassigned successfully" }
```

19. Payment APIs

These APIs handle cash and Razorpay payments.

POST /api/payments/razorpay/create-order

Purpose: Create a Razorpay payment order.

Request body:

```
{ "orderId": 101, "amount": 440 }
```

Response 200:

```
{
  "success": true,
  "data": {
    "razorpayOrderId": "order_abc123",
    "amount": 44000,
    "currency": "INR"
  }
}
```

POST /api/payments/razorpay/verify

Purpose: Verify Razorpay signature and mark order paid.

Request body:

```
{
  "orderId": 101,
  "razorpayOrderId": "order_abc123",
  "razorpayPaymentId": "pay_xyz123",
  "razorpaySignature": "signature_here"
}
```

Response 200:

```
{ "success": true, "message": "Payment verified successfully" }
```

PATCH /api/payments/:orderId/cash-paid

Purpose: Mark cash payment as completed.

Request body:

```
{ "collectedBy": 5 }
```

Response 200:

```
{ "success": true, "message": "Cash payment marked as paid" }
```

GET /api/payments/:orderId

Purpose: Get payment details for one order.

Response 200:

```
{
  "success": true,
  "data": {
    "paymentId": 20,
    "orderId": 101,
    "method": "cash",
    "amount": 440,
    "status": "pending"
  }
}
```

GET /api/payments

Purpose: Get all payments.

Response 200:

```
{ "success": true, "data": [] }
```

PATCH /api/payments/:paymentId

Purpose: Update payment status manually.

Request body:

```
{ "status": "success" }
```

Response 200:

```
{ "success": true, "message": "Payment updated successfully" }
```

DELETE /api/payments/:paymentId

Purpose: Delete a payment record.

Response 200:

```
{ "success": true, "message": "Payment record deleted successfully" }
```

20. Review APIs

These APIs store customer feedback after the order is finished.

POST /api/reviews

Purpose: Create a customer review.

Request body:

```
{
 "orderId": 101,
 "rating": 5,
 "comment": "Food was good and served quickly"
}
```

Response 201:

```
{
 "success": true,
 "message": "Review submitted successfully",
 "data": { "reviewId": 1 }
}
```

GET /api/reviews

Purpose: Get all reviews.

Response 200:

```
{ "success": true, "data": [] }
```

GET /api/reviews/:reviewId

Purpose: Get one review.

Response 200:

```
{
 "success": true,
 "data": {
  "reviewId": 1,
  "orderId": 101,
  "rating": 5,
  "comment": "Food was good"
 }
}
```

PATCH /api/reviews/:reviewId

Purpose: Update a review.

Request body:

```
{
 "rating": 4,
 "comment": "Food was good but slightly late"
}
```

Response 200:

```
{ "success": true, "message": "Review updated successfully" }
```

DELETE /api/reviews/:reviewId

Purpose: Delete a review.

Response 200:

```
{ "success": true, "message": "Review deleted successfully" }
```

21. Notification APIs

These APIs store and manage unread/read notifications.

GET /api/notifications

Purpose: Get notifications for the logged-in user.

Response 200:

```
{ "success": true, "data": [] }
```

GET /api/notifications/:notificationId

Purpose: Get one notification.

Response 200:

```
{
  "success": true,
  "data": {
    "notificationId": 1,
    "type": "new_order",
    "message": "New order assigned"
  }
}
```

POST /api/notifications

Purpose: Create a notification record.

Request body:

```
{
  "userId": 5,
  "orderId": 101,
  "type": "assigned",
  "message": "You have been assigned a new order"
}
```

Response 201:

```
{ "success": true, "message": "Notification created successfully" }
```

PATCH /api/notifications/:notificationId/read

Purpose: Mark a notification as read.

Response 200:

```
{ "success": true, "message": "Notification marked as read" }
```

DELETE /api/notifications/:notificationId

Purpose: Delete a notification.

Response 200:

```
{ "success": true, "message": "Notification deleted successfully" }
```

22. Dashboard and Report APIs

These APIs provide summary data for each role.

GET /api/dashboard/kitchen

Purpose: Get kitchen dashboard counts.

Response 200:

```
{
  "success": true,
```

```
  "data": {
    "pendingOrders": 5,
    "preparingOrders": 3,
    "readyOrders": 2
  }
}
```

GET /api/dashboard/waiter

Purpose: Get waiter dashboard counts.

Response 200:

```
{
  "success": true,
  "data": {
    "assignedOrders": 4,
    "servedOrders": 10,
    "pendingCashCollection": 2
  }
}
```

GET /api/dashboard/admin

Purpose: Get admin dashboard summary.

Response 200:

```
{
  "success": true,
  "data": {
    "totalOrders": 120,
    "totalRevenue": 45000,
    "pendingPayments": 7
  }
}
```

GET /api/reports/orders?from=YYYY-MM-DD&to=YYYY-MM-DD

Purpose: Get order report for a date range.

Response 200:

```
{ "success": true, "data": [] }
```

GET /api/reports/revenue?from=YYYY-MM-DD&to=YYYY-MM-DD

Purpose: Get revenue report for a date range.

Response 200:

```
{
  "success": true,
  "data": {
    "totalRevenue": 50000
  }
}
```

23. Socket Events

These events are used for live updates without page refresh.

Event	When it is sent	Who receives it
new_order	When customer places an order	Kitchen, admin
order_accepted	When chef accepts the order	Customer, admin

order_status_changed	When status changes	Customer, kitchen, waiter
waiter_assigned	When waiter is assigned	Waiter, admin
payment_updated	When payment is completed or changed	Customer, waiter, admin
notification_sent	When a new notification is created	Relevant user

24. Access Control Rules

These rules define which role can do which action.

- Admin: full control over users, roles, tables, categories, menu items, reports.
- Chef: can view orders, accept orders, update preparation status, and mark food prepared.
- Waiter: can view assigned orders, acknowledge assignment, serve orders, and collect cash.
- Customer: can place order, track order, pay online, and submit review.

25. Common HTTP Status Codes

These are the main response codes used across the system.

Code	Meaning	When used
200	OK	Request completed successfully
201	Created	New record created
400	Bad Request	Validation failed or bad input
401	Unauthorized	Missing or invalid token
403	Forbidden	User not allowed to do the action
404	Not Found	Resource not found
409	Conflict	Duplicate record exists
500	Internal Server Error	Unexpected server problem

26. Build Order

Recommended development sequence.

- Create database tables and foreign keys
- Build authentication and JWT protection
- Build menu, category, and customization APIs
- Build order creation and order item APIs
- Add order status logs and chef actions
- Implement waiter assignment logic
- Integrate Razorpay and cash payment flow
- Add review and notification APIs
- Add Socket.IO live updates
- Build dashboards for customer, chef, waiter, and admin
- Test the full end-to-end flow

27. Final Summary

This project is a real restaurant workflow system. It is stronger than a simple ordering app because it includes structured SQL design, role-based access, live order updates, waiter assignment, payments, reviews, and dashboards. It is suitable as an intermediate to advanced full-stack project.